



FIAP GRADUAÇÃO

CHALLENGE 2026

2º ANO

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Turmas de Fevereiro – 2º Semestre

CRONOGRAMA

2º SEMESTRE

CRONOGRAMA

DATA	EVENTO	STEAKHOLDER
21/08/2026	Mentoria Clyvo - Presencial	Clyvo
12/09/2026	ENTREGA DA SPRINT 3	ALUNO
26/09/2026	Feedback das entregas SPRINT 3	PROFESSORES
Setembro*	Pré-Banca de Professores – Seleção dos Projetos	PROFESSORES
Setembro*	Apresentação Banca Final Clyvo	Clyvo
04/11/2026	ENTREGA DA SPRINT 4	ALUNO
11/11/2026	Feedback das entregas SPRINT 4	PROFESSORES
24/10/2026	NEXT	FIAP

APRESENTAÇÕES – 2º SEMESTRE

- **A mentoria da Clyvo** têm como objetivo fornecer orientações e direcionamentos para o desenvolvimento dos projetos.
- **A pré-banca de professores** será voltada à avaliação dos projetos que estarão aptos para a apresentação final à Clyvo.
- **A apresentação final para a Clyvo** servirá para selecionar os melhores projetos, que participarão do evento NEXT e concorrerão à premiação na competição.

3º ENTREGAS

POR DISCIPLINA

ADVANCED BUSINESS DEVELOPMENT WITH .NET

Você deverá evoluir a aplicação ASP.NET Core desenvolvida nas sprints anteriores, adicionando camadas de monitoramento, observabilidade e testes automatizados. Esta sprint tem foco nos seguintes tópicos:

- Monitoramento e Observabilidade;
- Testes Automatizados com padrão AAA;
- Atualização da documentação do projeto.

Requisitos

1.Implementação de Monitoramento e Observabilidade (40 pts)

- Health Checks - Implementar endpoints de Health Check para verificar a saúde da API, conectividade com o banco de dados (Oracle/MongoDB) e disponibilidade de serviços externos. Utilizar Microsoft.Extensions.Diagnostics.HealthChecks. (15 pts)
- Logging Estruturado - Configurar logging estruturado com Serilog ou NLog, incluindo níveis de log (Information, Warning, Error), correlação de requisições e saída para console/arquivo. (10 pts)
- Tracing e Métricas: Implementar distributed tracing com OpenTelemetry ou Application Insights para rastrear requisições entre camadas da aplicação. Expor métricas de desempenho (tempo de resposta, taxa de erros). (15 pts)

ADVANCED BUSINESS DEVELOPMENT WITH .NET

2. Implementação de Testes Automatizados – Padrão AAA (50 pts)

- Testes Unitários com xUnit: Implementar testes unitários seguindo o padrão AAA (Arrange, Act, Assert) para as camadas de Domínio e Aplicação. Utilizar xUnit como framework de testes, com Moq ou NSubstitute para mocking de dependências. (20 pts)
- Testes de Integração: Implementar testes de integração para os endpoints da API utilizando WebApplicationFactory. Validar o fluxo completo de requisições HTTP, incluindo autenticação, respostas de sucesso e tratamento de erros. (15 pts)
- Cobertura e Organização dos Testes: Organizar os projetos de teste separados por camada (Unit, Integration). Garantir nomenclatura consistente dos testes (MetodoTestado_Cenario_ResultadoEsperado). Utilizar Fixtures e Collection Fixtures para compartilhar contexto entre testes. (15 pts)

3. Atualização do README (10 pts)

- Documentar os endpoints de Health Check e como monitorar a aplicação.
- Incluir instruções de como executar os testes (dotnet test).
- Atualizar a descrição geral do projeto com as novas funcionalidades.

COMPLIANCE, QUALITY ASSURANCE & TESTS

Desenvolva o plano detalhado do seu projeto Challenge, usando o AZURE BOARDS, contendo:

- Estrutura de trabalho em forma de Backlog de Produto, com detalhes de Épicas, Features e Product Backlog Items, no padrão Scrum (peso 20%);
- Documentação de descrição e critérios de aceite, explicando as histórias de usuário formadas pelos Épicas, Features e Product Backlog Items (peso 20%);
- Apontamento da Prioridade, Esforço, Dependência entre itens do backlog, ordenando o backlog na sequência de implementação e entrega (peso 20%);
- Release plan, contendo a indicação de qual a Sprint que desenvolverá e entregará cada item do backlog, criando um roadmap de entregas, balanceando os pontos de esforço entre Sprints (peso 20%);
- Detalhamento da Sprint atual do projeto (em curso), com Tarefas detalhadas com suas descrições, pontuações de esforço e dependências técnicas (20%).

Entregue o link de acesso ao seu plano de projetos na nuvem.

Garanta que o professor esteja cadastrado como membro da organização e administrador do projeto.

Entregas fora das especificações implicarão em perda de 10% na nota de avaliação.

DEVOPS TOOLS & CLOUD COMPUTING 1/7

Objetivo

Nesta Sprint, sua equipe deve implementar uma solução baseada em uma das disciplinas a seguir: **JAVA ADVANCED** OU **ADVANCED BUSINESS DEVELOPMENT WITH .NET**

A solução deve ser implementada junto com um banco de dados na nuvem

Escolha uma das opções abaixo:

Opção 1: ACR + ACI

Utilize Azure Container Registry (ACR) para armazenar sua imagem Docker e Azure Container Instance (ACI) para executar o container. Ambos devem ser utilizados nessa opção. Solução containerizada completa: App e Banco

Opção 2: Serviço de Aplicativo (App Service)

Publique sua aplicação em um App Service na Azure, com o banco de dados também na nuvem - Aqui nada pode ser Containerizado

DEVOPS TOOLS & CLOUD COMPUTING 2/7

Requisitos Obrigatórios

ATENÇÃO: NÃO PODE MISTURAR AS OPÇÕES DE ESCOLHA: Ou ACR/ACI ou Serviços de Aplicativos com Banco PaaS

-> Não será aceito Banco em Container com Serviço de Aplicativo ou ACI (App) com Banco PaaS por exemplo. Ou realiza a Containerização completa do App e Banco ou utilize Serviços de Aplicativos com os possíveis Bancos PaaS

Sua entrega deve conter os itens abaixo, independentemente da opção escolhida:

1. Descrição da Solução: explique brevemente o que a aplicação faz (inclua no README)
2. Descrição dos Benefícios para o Negócio: explique quais problemas a solução resolve ou quais melhorias ela traz (inclua no README)
3. Banco de Dados em Nuvem (obrigatório):
 - 3.1 Não será aceito o banco H2
 - 3.2 Serão aceitos Bancos em Containers na nuvem ou Bancos com serviços de PaaS. Por exemplo: Oracle (Container em Nuvem ou Banco da FIAP), MySQL, Azure SQL ou PostgreSQL
 - 3.3 Incluir o DDL das tabelas (tabelas, colunas, chave primária, comentários etc) criado em arquivo de texto separado com o nome: **script_bd.sql** com estrutura e comentários
 - 3.4 Utilize tabelas que representem o CORE da aplicação, nada de cadastro de cidade e estado, usuário e acesso etc. **Utilize tabelas significativas para a solução**

DEVOPS TOOLS & CLOUD COMPUTING 3/7

4. Implementar um CRUD completo (Inclusão, Alteração, Exclusão e Consulta) sobre **ao menos duas tabelas** da aplicação relacionadas entre si
5. Inserir e manipular **pelo menos 2 linhas com conteúdo significativo** nessas tabelas
6. Código-fonte publicado no GitHub

7. Arquivo PDF contendo:

Nome completo e RM de todos os integrantes

Link do repositório no GitHub

Link do vídeo no Youtube

Não pode ter mais nada no PDF, todo o resto é no GitHub

8. Requisitos Específicos Por Tipo De Entrega

Se escolher ACR + ACI:

- 8.1. Todos os recursos (App e Banco de Dados) devem ser criados via Azure CLI
- 8.2. O container do App não pode rodar como root ou admin (com privilégios administrativos)
- 8.3. Pode usar Dockerfile ou Docker Compose
- 8.4. Entregue todos os scripts do build e execução da imagem, como: Dockerfile, docker-compose.yml (se for o caso), Comandos utilizados: docker build, docker push, docker run etc. (README)

DEVOPS TOOLS & CLOUD COMPUTING 4/7

Se escolher Serviço de Aplicativo (App Service):

- 8.5. Todos os recursos (App e Banco de Dados) devem ser criados via Azure CLI
- 8.6. Entregue todos os scripts dos recursos criados na Azure, como: Grupo de recurso, Plano do serviço, Serviço de aplicativo, Banco de dados e configurações adicionais

9. Critérios de Avaliação (Pontuação)

9.1. Desenho da arquitetura da solução proposta com fluxos, recursos e explicação do funcionamento, baseado na disciplina de DevOps Tools e Cloud Computing (**até 20 pontos**)

9.2 Vídeo Demonstrativo da Solução (**até 80 pontos**):

Gravar um vídeo (mínimo 720p, com áudio claro e explicação por voz, sem uso de legendas nessa entrega) mostrando todo o funcionamento da solução, incluindo:

- Mostrar criando os recursos na Azure
- Clone do repositório no GitHub -> Obrigatório começar assim os testes da solução
- Deploy da aplicação seguindo exatamente os passos descritos no README.md
- Criação, configuração e testes do App e do Banco de Dados na nuvem seguindo exatamente os passos descritos no README.md
- Sem cortes no vídeo quando estiver evidenciando os testes da solução e a persistência de dados

DEVOPS TOOLS & CLOUD COMPUTING 5/7

9.3 Demonstração detalhada e individual de todas as operações do CRUD diretamente no Banco de Dados por SELECT:

Inserção de um registro → exibir no banco

Atualização do registro → exibir no banco

Exclusão do registro → exibir no banco

Consulta de registros

Evidenciar claramente a integração total entre o App e o Banco em nuvem, com tudo funcionando 100%

Observações:

Não há limite de tempo para o vídeo, mas evite excesso de duração

A apresentação deve ser clara, organizada e completa, evidenciando todas as etapas do processo

Lembre-se: O vídeo é sua Prova entregue, deve estar impecável para uma nota máxima

DEVOPS TOOLS & CLOUD COMPUTING 6/7

Penalidades (Descontos)

Entrega em LOCALHOST: Não será possível a correção - Zero de nota

Entrega fora da data e horário proposto no documento: Não será possível a correção - Zero de nota

Professor sem acesso ao Repositório ou Vídeo - Não será possível a correção - Zero de nota

Sem descrição da solução: -10 pontos

Sem benefício para o negócio: -10 pontos

Sem README.md com orientações de deploy/teste (How To): -30 pontos

Sem evidência clara **de cada operação CRUD** no banco de dados: -30 pontos

Vídeo com baixa qualidade ou sem explicação falada: -30 pontos

Sem o código fonte: -40 pontos

Sem o PDF com nome completo, RM e links: -30 pontos

Não incluir o DDL das tabelas: -10 pontos

Não inserir e manipular pelo menos 2 linhas com conteúdo significativo nas duas tabelas: -20 pontos

Utilizar apenas uma tabela no CRUD: -20 pontos

Utilizar tabelas que não fazem parte do CORE da solução: -30 pontos

Misturar as opções de entrega: -40 pontos

Deixar dados sensíveis expostos (usuário, senha e tokens por exemplo) no código fonte: -20 pontos

Desenho de arquitetura parecido com Fluxo, Togaf, UML não será aceito: -20 pontos

Utilizar um banco não permitido (vide lista): -40 pontos

DEVOPS TOOLS & CLOUD COMPUTING 7/7

Para opção ACR + ACI:

Recursos não criados via CLI: -30 pontos

Container do App com privilégio admin: -10 pontos

Script faltando: -10 pontos por script

Banco não Containerizado: -40 pontos

App não Containerizado: -40 pontos

Para opção Serviços de Aplicativos e Banco PaaS:

Recursos não criados via CLI: -30 pontos

Script faltando: -10 pontos por script

Banco Containerizado: -40 pontos

App Containerizado: -40 pontos

DISRUPTIVE ARCHITECTURES: IOT, IOB & GENERATIVE IA

Objetivos Principais

- Definir e documentar o componente de Inteligência Artificial que será integrado à solução proposta para a CLYVO VET, incluindo o problema a ser resolvido, os dados necessários, a estratégia de personalização e a arquitetura de integração com a aplicação.
- Gravar um vídeo pitch de aproximadamente 5 minutos apresentando a proposta da solução, o papel da IA no sistema, os benefícios para o tutor e para a clínica, e a arquitetura de funcionamento.

Objetivos Específicos

- Definir o problema de negócio que será tratado pela IA dentro da jornada contínua de cuidado do pet.
- Demonstrar como a IA contribuirá para personalização, priorização de ações, recomendação de serviços ou apoio à tomada de decisão.
- Descrever o fluxo de dados entre usuários, aplicação, banco de dados e componentes de IA.
- Identificar e documentar os dados necessários para alimentar a IA, incluindo origem, estrutura e utilização.
- Escolher e justificar a abordagem de IA adotada (IA Generativa, LLM, sistema de recomendação, motor de regras inteligentes, NLP ou modelo preditivo).

DISRUPTIVE ARCHITECTURES: IOT, IOB & GENERATIVE IA

Tarefas

- Definir qual problema será resolvido pela IA e como ela agregará valor à experiência do tutor, da clínica e do pet.
- Escolher a abordagem de IA mais adequada ao projeto e justificar tecnicamente sua utilização.
- Identificar e descrever os dados utilizados pela solução, incluindo perfil do pet, histórico clínico, vacinas, consultas, medicamentos, comportamento e demais informações relevantes.
- Criar um diagrama arquitetural mostrando a comunicação entre a aplicação, banco de dados, APIs e componentes de IA.

DISRUPTIVE ARCHITECTURES: IOT, IOB & GENERATIVE IA

CrITÉRIOS de Avaliação

- (até 60 pontos) Aplicação técnica de conceitos de IA.
- (até 20 pontos) Clareza e didática da apresentação em vídeo
- (até 20 pontos) Organização do repositório e documentação técnica.

Entregáveis obrigatórios:

- Arquivo .zip contendo:
- Link do vídeo publicado no YouTube (em modo não listado) com a apresentação da ideia central do projeto, demonstração funcional (real ou simulada) dos principais recursos implementados.
- Link para o repositório no GitHub, apresentando código-fonte e estrutura do projeto e o README com instruções de uso, tecnologias utilizadas e resultados parciais.

Condições de entrega

- A integridade e o conteúdo do arquivo entregue são de responsabilidade dos integrantes do grupo.
- Arquivos entregues sem conteúdo ou com arquivos corrompidos não serão considerados. Sugestão: confira seu anexo antes de publicar.
- Não serão aceitos arquivos enviados pelo Teams ou fora do prazo.

JAVA ADVANCED 1/3

Você deverá desenvolver uma aplicação web completa utilizando o framework Spring Boot para dar suporte à solução proposta. Esta aplicação deve ter o foco nos seguintes tópicos:

1. Camada de visualização (frontend);
2. Flyway para controle de versões do banco de dados;
3. Spring Security para autenticação e controle de acesso.

Requisitos

A aplicação deverá conter obrigatoriamente os seguintes elementos técnicos:

1. Frontend (30 pontos)
2. Flyway (20 pontos)
3. Spring Security (30 pontos)
 - Implementação de pelo menos dois tipos de usuário, com permissões diferentes.
 - Proteção de rotas com base no perfil do usuário.
4. Funcionalidades completas (20 pontos)
 - Implementação funcional de pelo menos dois fluxos completos do sistema (exceto CRUD)
 - Validações básicas nos formulários e nos dados.

JAVA ADVANCED 2/3

Penalidades

As penalidades abaixo serão aplicadas independentemente dos requisitos técnicos, e dizem respeito à qualidade do código e funcionamento geral do sistema:

Problema	Desconto
Violação evidente de princípios SOLID (ex: métodos gigantes, responsabilidades múltiplas)	-10 pontos por ocorrência
Código com repetições desnecessárias (violação de DRY)	-5 pontos por ocorrência
Código com problemas de legibilidade (violação de Clean Code)	-5 pontos por ocorrência
Comentários no lugar de refatorações claras	-3 pontos
Funcionalidade com comportamento inesperado ou erro evidente	-5 pontos por funcionalidade
Página que não carrega ou link quebrado	-5 pontos por página/link
Apresentação incompleta ou incoerente com a proposta do challenge.	-15 pontos

JAVA ADVANCED 3/3

Entrega

Os seguintes artefatos devem ser entregues no portal até o prazo definido. Não serão aceitas entregas após o prazo ou por outros meios.

- Link do repositório público do GitHub com o código fonte da aplicação Spring completa.
- O readme do repositório deve conter todas as instruções necessárias para a instalação, execução e acesso da aplicação.
- Vídeo demonstração da aplicação funcionando com apresentação das principais funcionalidades da aplicação web (máx. 10 min.)

Avaliação Oral

Após a entrega do projeto, cada aluno participará de uma avaliação oral individual em sala de aula. Essa etapa complementa a avaliação técnica. O objetivo é verificar a compreensão do código entregue e promover o uso consciente de ferramentas como a inteligência artificial. Durante essa conversa, o aluno deverá:

- Explicar trechos específicos do próprio código;
- Justificar decisões de implementação;
- Comentar eventuais dificuldades encontradas;
- Descrever se e como utilizou ferramentas de IA no processo

MASTERING RELATIONAL AND NON-RELATIONAL DATABASE

- Demanda: Desenvolver 02 procedimentos, 02 funções, 01 gatilho (trigger) e os scripts completos de estrutura e carga de dados utilizando o banco de dados relacional Oracle. A atividade também exige a entrega de documentação técnica com prints de execução e tratamento de exceções implementado em formato .PDF e .SQL.
- O trabalho tem como foco:
- Desenvolver lógica procedural SQL estruturada
- Manipular dados relacionais e convertê-los para JSON sem uso de funções automáticas;
- Implementar auditoria via trigger;
- Aplicar boas práticas de documentação, tratamento de erros e organização de código.

MASTERING RELATIONAL AND NON-RELATIONAL DATABASE

A. Especificações:

Procedimentos (30 pontos)

Procedimento 1:

Realizar JOIN entre duas ou mais tabelas relacionais do seu projeto.

Exibir os dados no formato JSON (string).

A transformação relacional para JSON deve ser feita manualmente com uma função criada pelo grupo (ver Função 1).

Cada tabela usada deve conter no mínimo 5 registros válidos.

Tratar no mínimo 3 exceções distintas com EXCEPTION WHEN.

MASTERING RELATIONAL AND NON-RELATIONAL DATABASE

Especificações:

B. Procedimento 2:

Ler os dados de uma tabela de fatos que contenha, pelo menos, duas colunas categóricas (por exemplo, agencia e conta) e uma coluna numérica com valores a serem somados (por exemplo, saldo). Esse procedimento deve calcular e exibir:

Os valores somados por combinação completa das categorias (agencia, conta);

Um subtotal por grupo da primeira categoria (agencia);

Um total geral ao final da listagem.

As somas devem ser feitas diretamente dentro do procedimento, sem o uso de recursos automáticos do Oracle. Os grupos de subtotal devem estar na mesma coluna e linha da saída, com os valores de agrupamento ausentes (nulos) nos casos de subtotal e total geral.

A lógica de somatório deve ser manual, feita dentro do corpo do procedimento.

Não é permitido o uso de funções automáticas de agregação avançada como ROLLUP, CUBE, GROUPING SETS, GROUPING, ou semelhantes.

MASTERING RELATIONAL AND NON-RELATIONAL DATABASE

O procedimento deve conter tratamento de pelo menos 3 exceções distintas.

A tabela utilizada deve conter pelo menos 5 linhas detalhadas para que haja dados suficientes para os cálculos de subtotal e total geral.

A ordem de exibição deve seguir a hierarquia das categorias.

A exibição deve seguir a ordem de agrupamento e apresentar o resultado no seguinte formato:

Agencia	Conta	Saldo
-----	-----	-----
1	1	4363.55
1	2	4794.76
1	3	4718.25
1	4	5387.45
1	5	5027.34
Sub Total		24291.35
2	1	5652.84
2	2	4583.02
2	3	5555.77
2	4	5936.67
2	5	4508.74
Sub Total		26237.04
Total Geral		50528.39

MASTERING RELATIONAL AND NON-RELATIONAL DATABASE

A. Funções (30 Pontos)

Função 1:

Crie uma Função que recebe dados relacionais e retorna uma string no formato JSON.

A lógica de conversão deve ser desenvolvida pelo grupo.

Proibido o uso de funções internas/built-in do Oracle, como TO_JSON, JSON_OBJECT, JSON_VALUE, JSON_QUERY, JSON_TABLE, ou similares.

Tratar no mínimo 3 exceções distintas.

B. Função 2:

Crie uma Função que substitui algum processo lógico do projeto, como:

Validação de senha, cálculos matemáticos, verificação de limites, etc.

Tratar no mínimo 3 exceções distintas.

MASTERING RELATIONAL AND NON-RELATIONAL DATABASE

Trigger (30 pontos)

Trigger de Auditoria (DML)

Criar uma tabela de auditoria com os campos:

Nome do usuário

Tipo da operação (INSERT, UPDATE, DELETE)

Data e hora da operação

Valores anteriores (:OLD)

Valores novos (:NEW)

Criar uma trigger AFTER INSERT OR UPDATE OR DELETE ON <tabela> que grave esses dados em cada operação realizada.

MASTERING RELATIONAL AND NON-RELATIONAL DATABASE

Entrega e Documentação (10 pontos)

a) Arquivo PDF: 2TDSX_2024_Proj_BD.pdf

Capa com: Nomes completos dos integrantes, RMs, em ordem alfabética

Prints de tela que mostrem: Execução de cada função/procedimento/gatilho

Pelo menos uma exceção tratada por função e procedimento

Código comentado e bem organizado

Código da 2ª Sprint corrigido conforme feedback anterior

b) Arquivo SQL: 2TDSX_2024_CodigoSql_Integrantes.sql

Deve conter todo o código necessário para execução do projeto, incluindo:

CREATE TABLE de todas as tabelas utilizadas

INSERT INTO com registros (mínimo 5 por tabela)

Procedimentos (CREATE OR REPLACE PROCEDURE)

Funções (CREATE OR REPLACE FUNCTION)

Trigger de auditoria - Comentários no código indicando a finalidade de cada bloco

MASTERING RELATIONAL AND NON-RELATIONAL DATABASE

Infração	Descrição	Desconto
Uso de funções built-in para JSON	Uso de TO_JSON, JSON_OBJECT, JSON_VALUE, etc.	-10 pontos por ocorrência
Falta de tratamento de exceções	Item sem EXCEPTION WHEN ou equivalente	-5 pontos por item
Ausência de prints com exceções	Não apresentar erro tratado em execução	-5 pontos por item
Código sem comentários ou desorganizado	Dificulta leitura e correção	-5 pontos
Tabelas com menos de 5 registros	Não cumpre requisito mínimo	5 pontos por tabela
Trigger incompleta ou não funciona	Não registra auditoria corretamente	-10 pontos
Falta de documentação mínima	Sem capa, prints, ou explicações claras	-5 a -10 pontos
Não entregar o arquivo .sql completo	Ausência de scripts com CREATE, INSERT, etc.	-10 pontos

MOBILE APPLICATION DEVELOPMENT ^{1/12}

Esta é a entrega intermediária mais importante do projeto. Ao final desta sprint, o aplicativo deve estar funcional, permitindo que o problema escolhido pelo grupo seja efetivamente resolvido, ainda que de forma parcial.

O protótipo entregue na Sprint 1 não é mais aceitável como referência visual apenas: ele deve ter sido convertido em uma base real de desenvolvimento. A aplicação deve conter fluxos principais implementados, integração consistente com a API e uma interface utilizável, ainda que não finalizada.

Nesta etapa, espera-se que:

- As funcionalidades centrais do sistema estejam implementadas
- O app execute ações reais (criar, consultar, atualizar ou processar dados)
- A integração com a API funcione de ponta a ponta
- O código esteja organizado, legível e compatível com boas práticas
- A aplicação não seja apenas uma navegação de telas sem lógica real

A Sprint 3 não é fase de rascunho, protótipo navegável ou simulação. Ela representa a consolidação técnica do projeto, que será refinado e finalizado na Sprint 4

MOBILE APPLICATION DEVELOPMENT 2/12

Requisitos Avaliativos

- Navegação entre telas (5 pontos)
- Integração com API backend (HTTP) (35 pontos)
- Sistema de autenticação (Login) (20 pontos)
- Arquitetura e organização do código (20 pontos)
- Documentação e apresentação da entrega (20 pontos)

Total: 100 pontos

MOBILE APPLICATION DEVELOPMENT 3/12

Navegação entre Telas (5 pontos)

A aplicação deve implementar navegação funcional entre telas, utilizando uma biblioteca de navegação apropriada e configuração explícita de rotas.

Critérios de Avaliação

- Quantidade mínima de telas (2 pontos)
 - O aplicativo deve possuir no mínimo 6 telas distintas
 - Telas duplicadas, telas vazias ou variações visuais da mesma tela não serão contabilizadas
 - Cada tela deve representar uma funcionalidade ou fluxo diferente do sistema
- Uso de biblioteca de navegação (2 pontos)
 - A navegação deve ser implementada utilizando uma biblioteca de navegação (ex: React Navigation ou expo router)
 - Não será aceito controle manual de telas via condicionais, if, useState ou renderização condicional como substituto de navegação, nem misturar o uso das bibliotecas
- Configuração correta das rotas (1 ponto)
 - As rotas devem estar explicitamente declaradas
 - A navegação entre telas deve ocorrer por meio dessas rotas
 - Navegação quebrada, rotas inexistentes ou não utilizadas implicam perda deste ponto

MOBILE APPLICATION DEVELOPMENT 4/12

Integração com API Backend HTTP (35 pontos)

A aplicação deve integrar-se a um backend acessado via HTTP, realizando operações reais de comunicação com a API definida pelo grupo.

Critérios de Avaliação

- Integração real com API HTTP (10 pontos)
 - As requisições devem ser implementadas com TanStack Query (useQuery, useMutation ou equivalentes)
 - Os dados exibidos na interface devem ser provenientes exclusivamente da API
 - Não será aceito uso de dados mockados, arquivos locais ou valores fixos no código
- Implementação de funcionalidades baseadas em API (5 pontos)
 - O aplicativo deve possuir no mínimo duas funcionalidades distintas dependentes da API
 - Cada funcionalidade deve utilizar dados reais retornados pelo backend e refletir alterações feitas pelo usuário
- CRUD completo integrado à interface (10 pontos)
 - As operações de Create, Read, Update e Delete devem estar implementadas nas duas funcionalidades
 - Todas as operações devem ser acessíveis por meio da interface do aplicativo
 - Não será aceito CRUD parcialmente implementado ou simulado
- Estados de carregamento e atualização de dados (10 pontos)
 - A aplicação deve exibir estado de carregamento (loading) durante requisições
 - Alterações nos dados devem refletir automaticamente na interface, sem necessidade de reiniciar o app ou recarregar manualmente

MOBILE APPLICATION DEVELOPMENT 5/12

Sistema de autenticação (Login) (20 pontos)

Funcionamento completo do fluxo de autenticação, clareza e segurança mínima no processo. Validação dos formulários, tratamentos de erros e indicadores de carregamento devem estar presentes. Pode utilizar Firebase Authentication ou API desenvolvida nas disciplinas de JAVA ou .NET

Critérios de Avaliação

- Autenticação real utilizando serviço externo (6 pontos)
 - O login deve ser implementado utilizando um serviço de autenticação real
 - É permitido o uso de Firebase Authentication
 - Não será aceito login baseado apenas em validações locais, comparações de string ou usuários fixos no código
- Fluxo completo de autenticação (6 pontos)
 - Tela de login e cadastro
 - Persistência da sessão do usuário autenticado
 - O usuário não deve precisar autenticar novamente ao reabrir o aplicativo
- Proteção de rotas e telas (4 pontos)
 - Telas protegidas devem ser acessíveis apenas após autenticação
 - O usuário não autenticado não deve conseguir acessar telas internas por navegação direta ou atalhos
 - O controle de acesso deve estar integrado ao sistema de navegação
- Logout funcional (4 pontos)
 - A aplicação deve permitir que o usuário encerre a sessão
 - Após o logout, o acesso às telas protegidas deve ser bloqueado imediatamente

MOBILE APPLICATION DEVELOPMENT 6/12

Arquitetura e Organização do Código (20 pontos)

O projeto deve apresentar uma arquitetura clara, organizada e coerente.

Critérios de Avaliação

- Separação de responsabilidades (6 pontos)
 - O código deve estar organizado separando claramente:
 - Interface (telas/componentes)
 - Lógica de negócio
 - Camada de acesso a dados / APIs
 - Não será aceito código de regra de negócio ou chamadas HTTP diretamente em componentes de tela
- Estrutura de pastas consistente (5 pontos)
 - O projeto deve possuir uma estrutura de pastas clara e padronizada
 - Arquivos não devem estar todos concentrados em uma única pasta
 - A organização deve permitir identificar facilmente onde estão telas, serviços, hooks e componentes reutilizáveis

MOBILE APPLICATION DEVELOPMENT 7/12

Arquitetura e Organização do Código (20 pontos)

O projeto deve apresentar uma arquitetura clara, organizada e coerente.

CrITÉRIOS de Avaliação

- Uso adequado de hooks e abstrações (5 pontos)
 - Lógica reutilizável deve estar abstraída em hooks ou serviços
 - Não será aceito código duplicado em múltiplas telas sem justificativa
 - Hooks como os do TanStack Query devem estar isolados da camada de UI
- Legibilidade e manutenibilidade do código (4 pontos)
 - Nomes de arquivos, funções e variáveis devem ser coerentes e autoexplicativos
 - Código excessivamente acoplado, confuso ou difícil de entender implicará perda de pontuação
 - O projeto deve permitir evolução sem necessidade de reescrita completa

MOBILE APPLICATION DEVELOPMENT 8/12

Documentação e Apresentação da Entrega (20 pontos)

A entrega deve conter documentação escrita e apresentação em vídeo, demonstrando claramente o funcionamento do aplicativo e o atendimento aos requisitos da Sprint 3.

Critérios de Avaliação

- **Documentação no README do repositório** (5 pontos)
 - Descrição do problema escolhido e da solução proposta
 - Tecnologias utilizadas e instruções para execução do projeto
- **Vídeo de apresentação do aplicativo** (15 pontos)
 - O grupo deve gravar um **vídeo com narração**, demonstrando o aplicativo em funcionamento
 - O vídeo deve ser **publicado no YouTube** e o link informado junto à entrega, no README.md
 - O vídeo deve apresentar, de forma clara e objetiva:
 - Navegação entre as telas
 - Sistema de autenticação
 - Integração com API backend
 - Comportamento do aplicativo em uso real (smartphone ou emulador)
 - A demonstração deve evidenciar explicitamente os **requisitos avaliativos da Sprint 3**
 - Duração máxima: **5 minutos**

MOBILE APPLICATION DEVELOPMENT 9/12

Penalidades

As penalidades abaixo serão aplicadas independentemente da pontuação obtida nos requisitos avaliativos, podendo resultar em redução significativa da nota final.

I. Não realizar a entrega via GitHub Classroom (-20 pontos)

A entrega deve ser feita exclusivamente pelo repositório oficial do GitHub Classroom.

II. Ausência de vídeo de apresentação (-20 pontos)

A não entrega do vídeo ou entrega em formato diferente do especificado implica penalização integral deste item.

III. Ausência ou inadequação do arquivo README.md (-10 pontos)

Projetos sem README ou com documentação claramente insuficiente estarão sujeitos à penalização.

IV. Aplicativo fora do escopo das aulas (-60 pontos)

Aplicações que utilizem soluções, frameworks, arquiteturas ou abordagens não vistas ou não autorizadas em aula, de forma a **simplificar artificialmente o desenvolvimento**.

V. Histórico de commits incoerente, inexistente ou artificial (-50 pontos)

É esperado que o repositório apresente:

- Evolução gradual do projeto
- Commits frequentes e coerentes
- Mensagens claras e descritivas

Repositórios com histórico artificial (ex: poucos commits grandes, commits genéricos, ou upload único do projeto) ou que não demonstrem envolvimento prático com o desenvolvimento poderão ser penalizados.

MOBILE APPLICATION DEVELOPMENT 10/12

VI. Integrações simuladas ou incompletas (-20 pontos)

Aplica-se quando for identificado qualquer um dos seguintes casos:

- Uso de API apenas para operações de leitura (GET)
- CRUD incompleto ou parcialmente simulado
- Código de integração não utilizado pela interface
- Dados retornados pela API ignorados ou substituídos por valores fixos ou mockados
- Necessidade de reiniciar o aplicativo para refletir alterações de dados
- Atualização manual da interface com useState após requisições

Esta penalidade cobre **todas as tentativas de “fingir integração”**.

VII. Navegação simulada ou inexistente (-10 pontos)

Aplica-se quando a navegação do aplicativo não for baseada em rotas reais.

Inclui, mas não se limita a:

- Alteração de componentes dentro da mesma tela como substituto de navegação
- Exibição/ocultação de interface sem troca de rota

VIII. Sistema de autenticação fictício ou incompleto (-20 pontos)

Aplica-se quando o sistema de autenticação não for real ou puder ser facilmente burlado.

Inclui:

- Login com usuário e senha fixos no código
- Controle de acesso feito apenas com variáveis locais ou useState
- Telas protegidas acessíveis sem autenticação
- Persistência de sessão inexistente ou simulada

MOBILE APPLICATION DEVELOPMENT 11/12

IX. Arquitetura inadequada ou código não manutenível (-30 pontos)

Aplica-se quando o projeto apresentar problemas estruturais graves, tais como:

- Toda a lógica concentrada em componentes de tela
- Um único arquivo contendo múltiplas responsabilidades
- Repetição excessiva de código sem abstração
- Estrutura que dificulte a leitura, manutenção ou evolução do projeto

X. Aplicativo não funcional (-100 pontos)

Será atribuída **nota zero na Sprint** caso o aplicativo:

- Não execute corretamente
- Não inicie (erro de build, crash ao abrir)
- Apresente falhas que impeçam a demonstração das funcionalidades avaliativas
- Não permita a execução dos fluxos principais do sistema durante a avaliação
- Dependendo de ajustes manuais, configurações externas não documentadas ou intervenção do professor para funcionar

MOBILE APPLICATION DEVELOPMENT 12/12

XI. Vídeo de apresentação incompatível com o aplicativo entregue (-100 pontos)

- Não demonstre o **aplicativo real entregue no repositório**
- Apresente protótipos ou telas de ferramentas como **Figma ou similares**
- Demonstre uma versão diferente da aplicação entregue
- Não permita verificar, de forma inequívoca, que o vídeo corresponde ao código avaliado
- O vídeo deve demonstrar **exclusivamente o aplicativo funcional**, em execução, conforme entregue para avaliação.

Observações Gerais

- As penalidades **são cumulativas**
- A aplicação de uma penalidade **não exige a ocorrência de todos os itens listados**
- A identificação de **um único caso representativo** é suficiente para aplicação
- Penalidades podem resultar em **nota final zero**, independentemente da pontuação obtida nos critérios avaliativos